# CIS 476
# Term Project

Professor Tommy Xu

Asiyah Shoeb

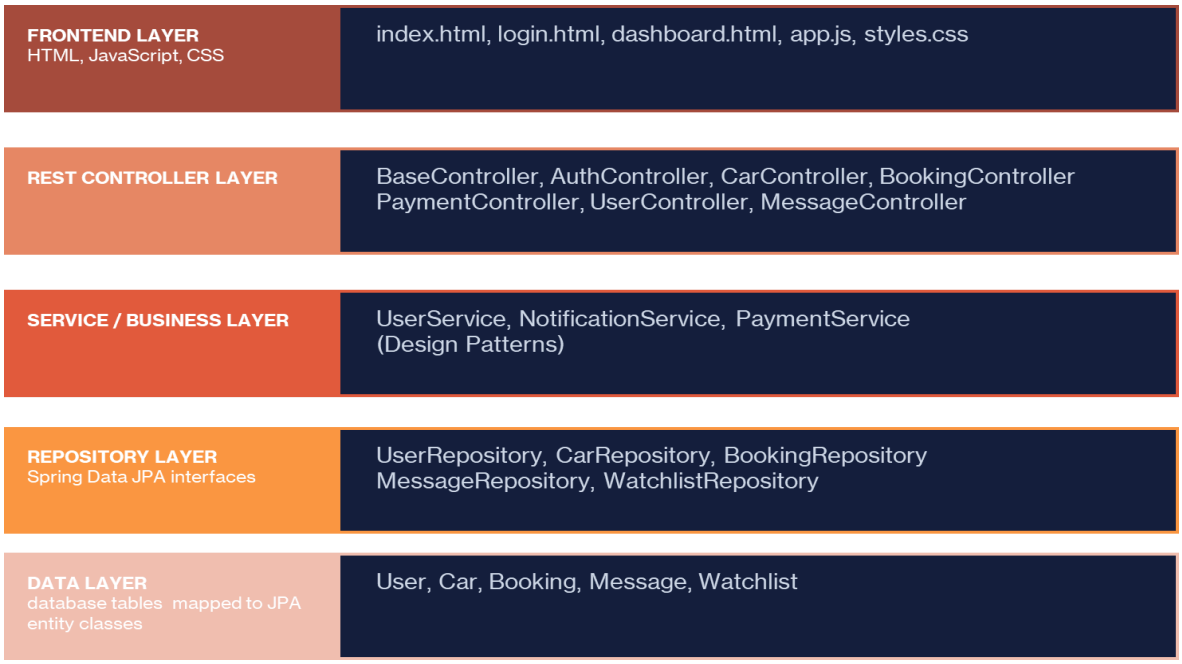
**1. Introduction** **3**
**2. Architecture** **3**
2.1. Model / Data Layer 4
2.2. Repository Layer 5
2.3. Service Layer 6
2.3.1. User Authentication and Encryption - Singleton Pattern 7
2.3.2. Observer Pattern for Booking Notifications 8
2.3.3. Mediator Pattern for UI Components 9
2.3.4. Builder Pattern for Car Listing Creation 10
2.3.5. Proxy Pattern for Payment Integration 11
2.3.6. Chain of Responsibility for Password Recovery 12
2.4. Controller / REST API Layer 13
2.5. DTO 14
**3. User-Interface Screenshots** **15**
3.1. User Registration and Authentication: 15
3.1.1. User Registration 15
3.1.2. Login 17
3.1.3. Forgot Password 18
3.2. Car Listing and Management (Owner): 21
3.2.1. Create Car Listing 21
3.2.2. Edit Car Listing 22
3.2.3. Overlapping Dates/Times Prevention 23
3.3 Search and Booking (Renter): 24
3.3.1. Search Car Listings 24
3.3.2. Watch Car Listings 26
3.3.3. Reserve Car 27
3.4. Messaging and Communication 29
3.4.1. Messaging System 29
3.4.2. Notification System 30
3.5. Payment 32
**4. References** **33**

# 1. Introduction

DriveShare is a web application that allows users to rent/host cars. This project was built with Spring Boot for backend and HTML/CSS/JavaScript for frontend. Persistent data is stored in an in-memory H2 database with Spring Data JPA.

# 2. Architecture

The architecture of DriveShare is organized into the following layers: frontend, rest controller, service/business, repository, and data.

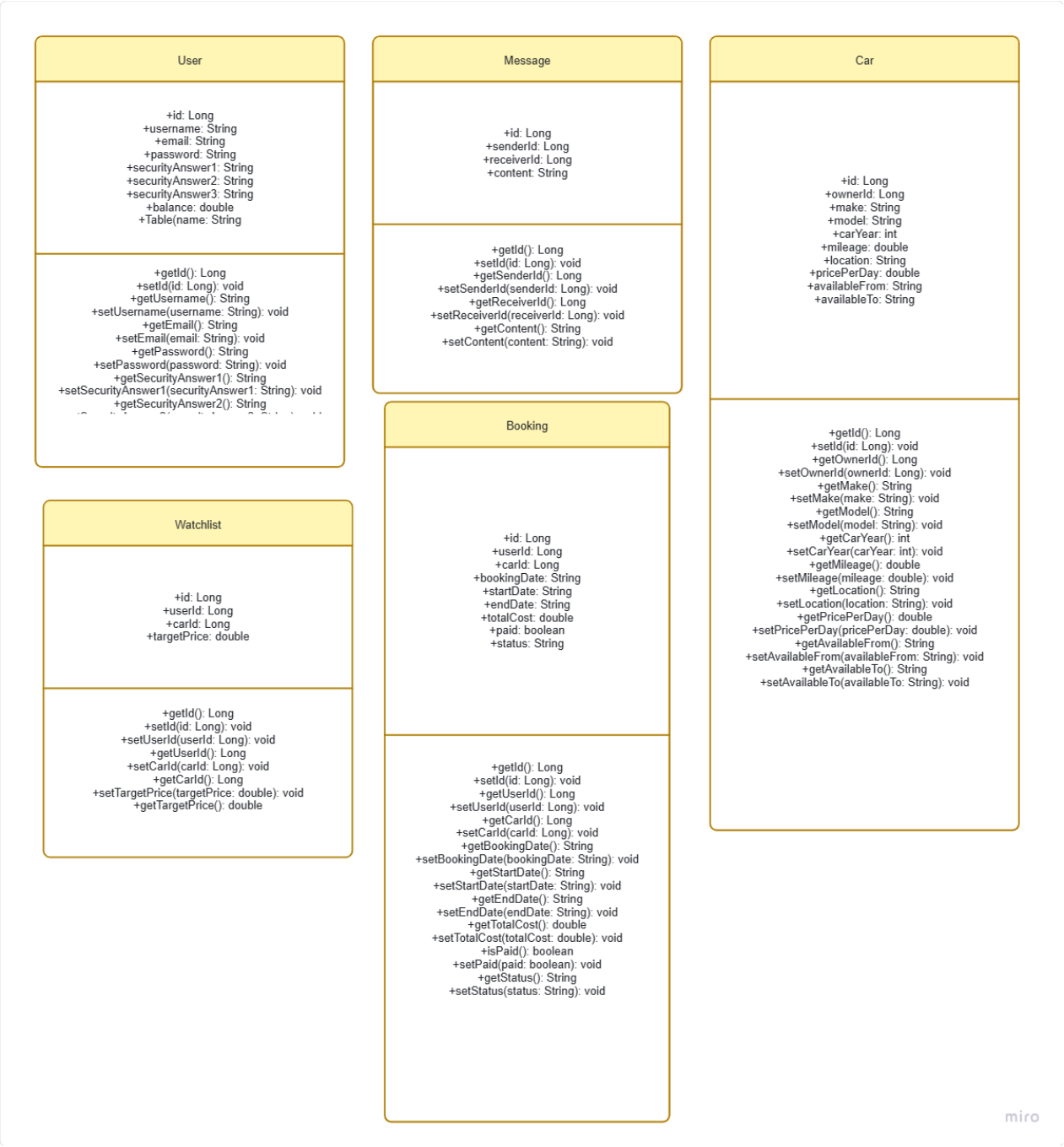



The frontend consists of static HTML/JS pages that make fetch() calls to the REST API with an X-Session-Token header to identify the logged-in user. Those requests hit a controller, which resolves the user from the session token via BaseController, validates input, and either handles the request directly or delegates to a service where the core business logic lives — for example, UserService runs the Chain of Responsibility for password reset. Services and controllers both call the repository layer, which are Spring Data JPA interfaces that abstract all database reads and writes. The objects flowing through all these layers are the models — JPA entities that map directly to database tables.

For example, when a renter books a car: dashboard.html sends POST /api/bookings. BookingController saves a Booking via BookingRepository, invoking UIMediator, which calls NotificationService that writes notifications, and the frontend polls and displays them.

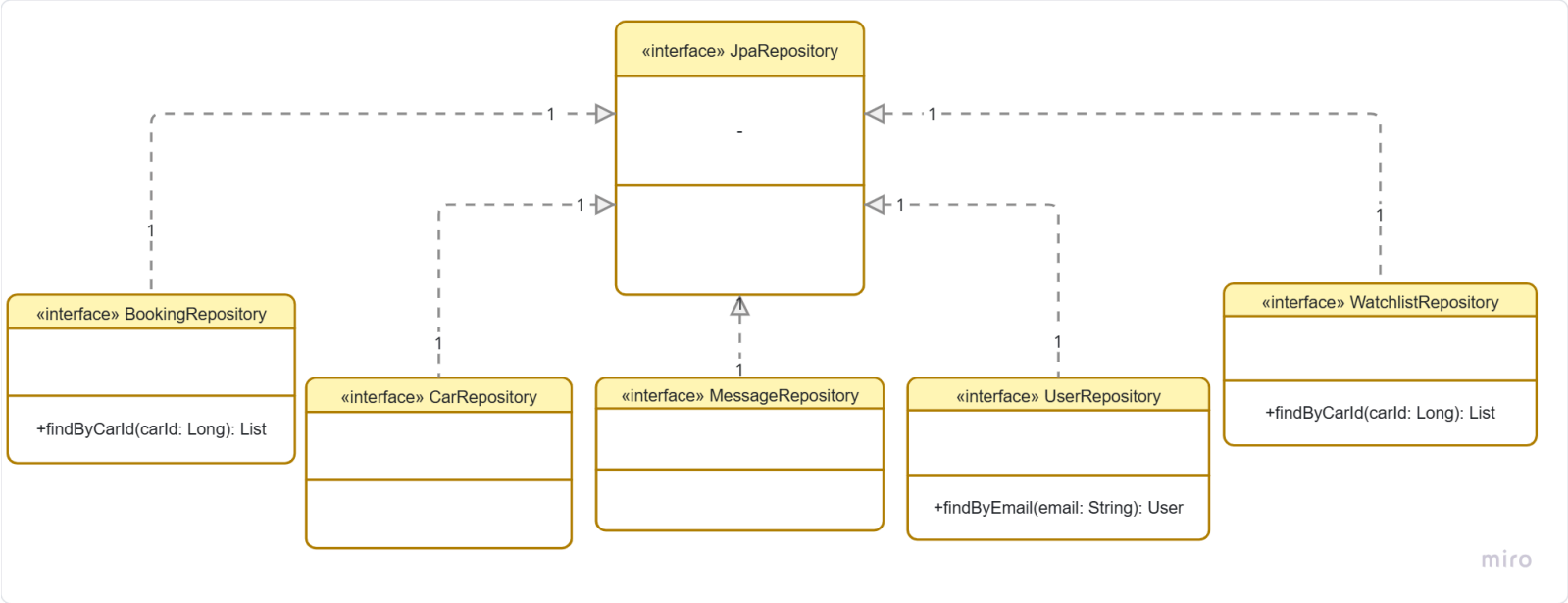
## 2.1. Model / Data Layer

**User.java:** Represents a registered user.
**Car.java:** Represents a car listing
**Booking.java:** Represents a rental request between a renter and a car.
**Message.java:** Represents a chat message between two users.
**Watchlist.java:** Represents a renter "watching" a car for price changes



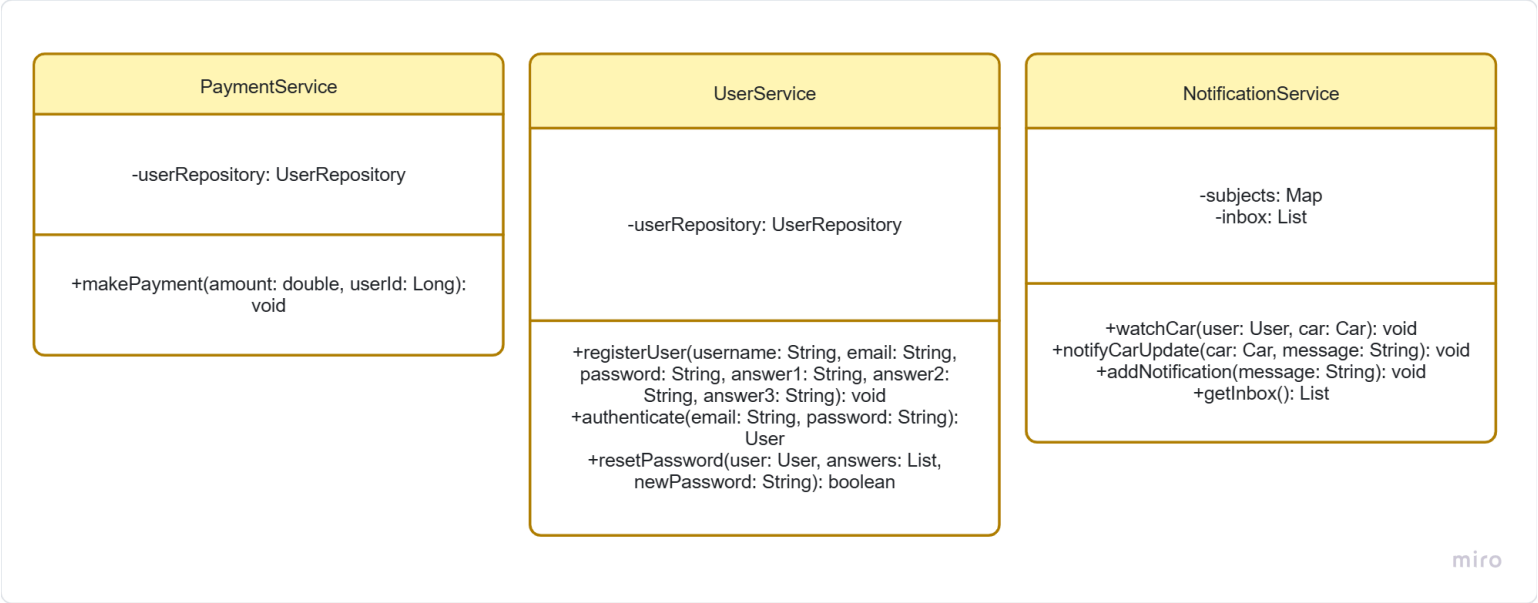
## 2.2. Repository Layer

**UserRepository.java:** Extends JpaRepository<User, Long>. Adds findByEmail(String) for login and password reset lookups.
**CarRepository.java:** Extends JpaRepository<Car, Long>. Provides standard CRUD for car listings
**BookingRepository.java:** Extends JpaRepository<Booking, Long>. Adds findByCarId(Long) to retrieve all bookings for a specific car (used for availability conflict checks).
**MessageRepository.java:** Extends JpaRepository<Message, Long>. Provides standard CRUD for chat messages.
**WatchlistRepository.java:** Extends JpaRepository<Watchlist, Long>. Adds findByCarId(Long) to retrieve all watchers of a car (used to trigger price-drop notifications).



## 2.3. Service Layer

**UserService.java:** Handles user registration (creates and persists a User), authentication (email/password match), and password reset by constructing and executing the Chain of Responsibility.
**NotificationService.java:** Maintains an in-memory inbox list of notification strings and a Map<Long, CarSubject> for the Observer pattern. Exposes watchCar(), notifyCarUpdate(), and addNotification().
**PaymentService:** Thin service layer for payment processing. Delegates to PaymentProxy, which performs a security check before forwarding to RealPayment.



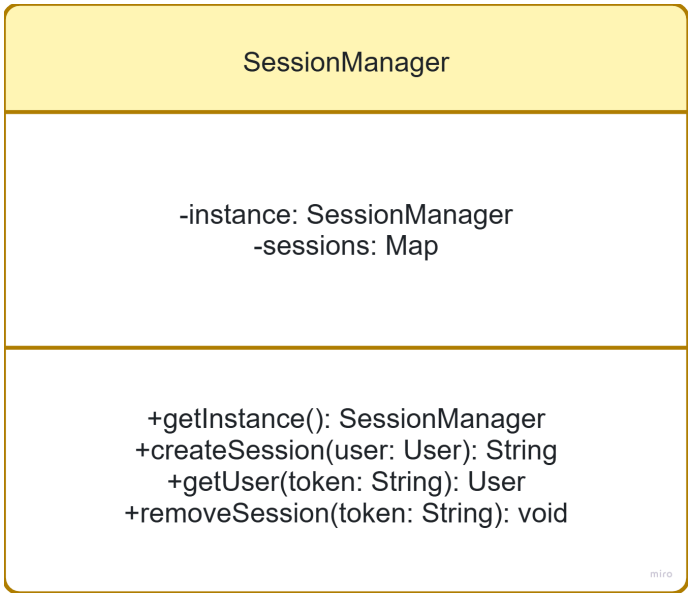
### 2.3.1. User Authentication and Encryption - Singleton Pattern

**SessionManager.java:**

- This class has a private constructor and a private static field holding the single instance.
- The getInstance() method double-checks to create the instance. The first check avoids locking overhead once the instance exists, and the second check inside the synchronized block prevents a condition where two threads simultaneously see null.
- Instead of holding a single User (which would limit the system to one logged-in user at a time), there is a ConcurrentHashMap<String, User> that maps session tokens to User objects.

**Relationships:**

- AuthController calls SessionManager.getInstance().createSession(user) on login and removeSession(token) on logout.
- BaseController.resolveUser(token) calls SessionManager.getInstance().getUser(token) on every API request to identify who is making the call. All other controllers extend BaseController and inherit this.



### 2.3.2. Observer Pattern for Booking Notifications

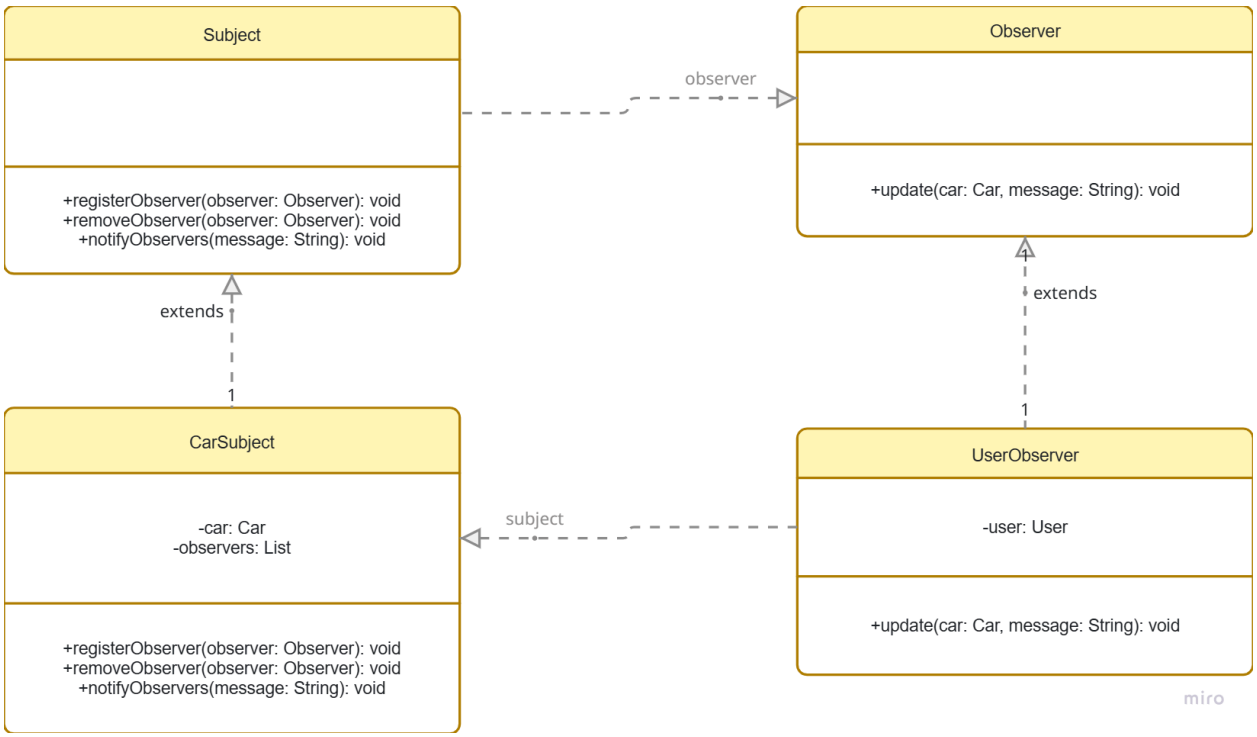
**Subject.java** (interface)
**Object.java** (interface)
**CarSubject.java** (concrete subject): It wraps a Car entity and maintains a List<Observer>. When notifyObservers(message) is called, it iterates the list and calls update(car, message) on each registered observer.
**UserObserver.java** (concrete observer): It wraps a User and implements update() by printing a notification string containing the user's email, the message, and the car model.

**Triggers:**

1. A car's price drops — CarController.updateCar() detects this and calls notificationService.notifyCarUpdate()
2. A car is booked — BookingController.createBooking() calls notifyCarUpdate() to alert watchers

**Relationships:**

- NotificationService contains Map<Long, CarSubject>, creates UserObserver instances
- CarController and BookingController both call NotificationService

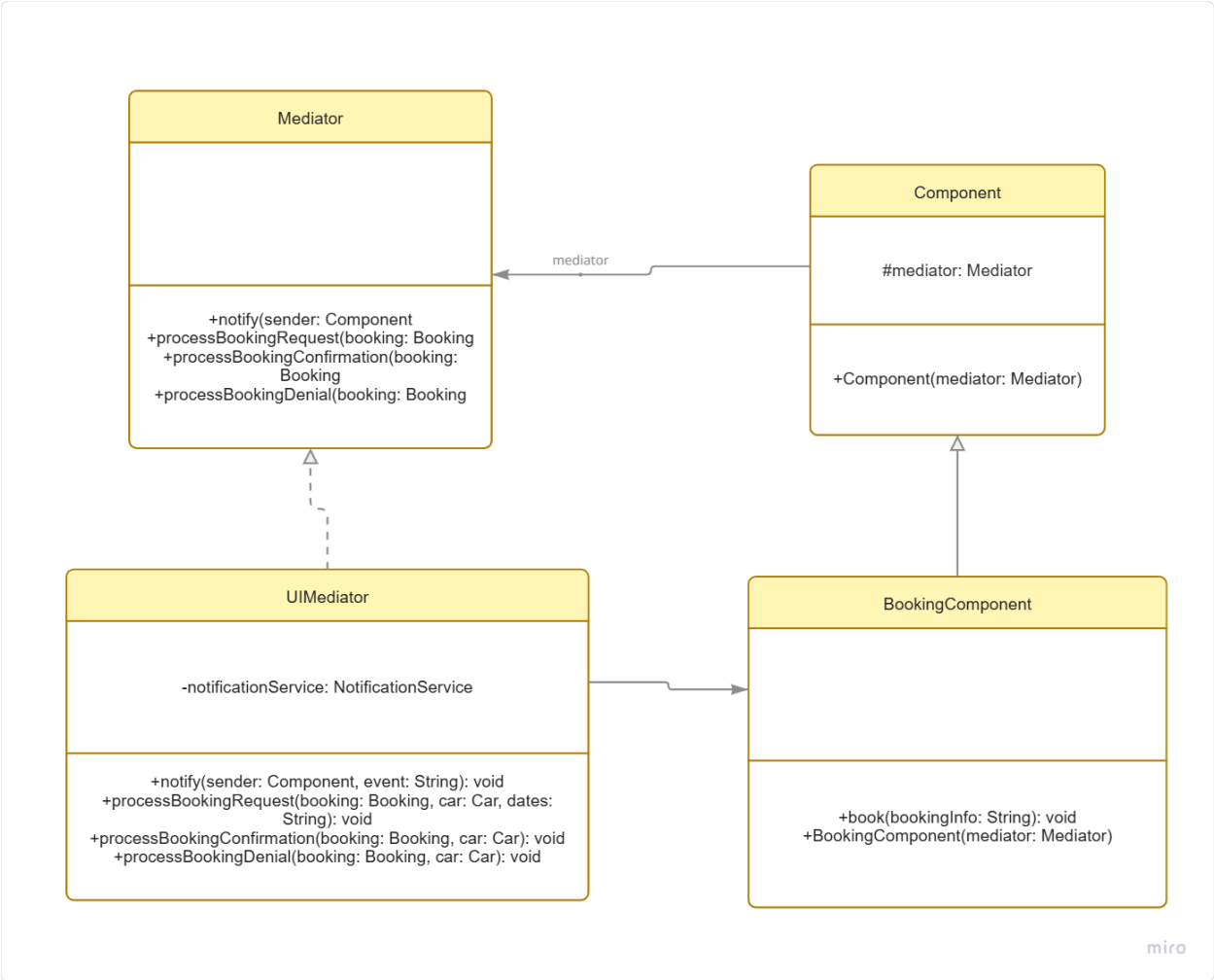


### 2.3.3. Mediator Pattern for UI Components

**Mediator.java** (Mediator Interface): Declares notify() for generic events plus three domain-specific coordination methods: processBookingRequest, processBookingConfirmation, and processBookingDenial.
**UIMediator.java** (Concrete Mediator):

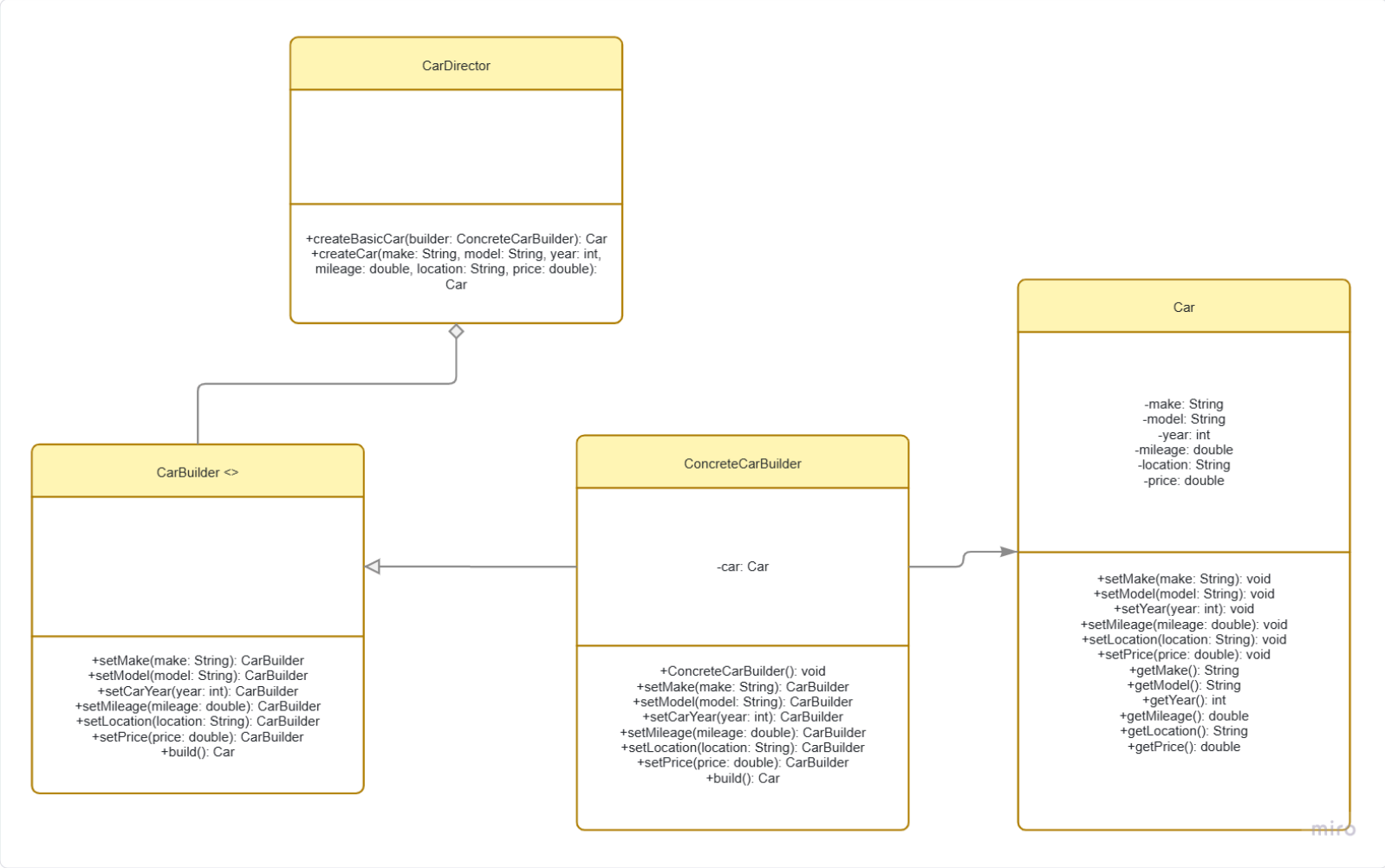
- processBookingRequest: logs the event via notify(), sends a notification to the owner (new request) and the renter (request submitted).
- processBookingConfirmation: logs via BookingComponent, then sends the renter a confirmation with the total cost and payment due message.
- processBookingDenial: logs the event and sends the renter a denial notification.

**Component.java** (Abstract Component): Holds a reference to Mediator.
**BookingComponent.java** (Concrete component): Sits between the controller and the mediator. When a booking action occurs, it calls mediator.notify() to signal the event, and the mediator handles all coordination from there.



### 2.3.4. Builder Pattern for Car Listing Creation

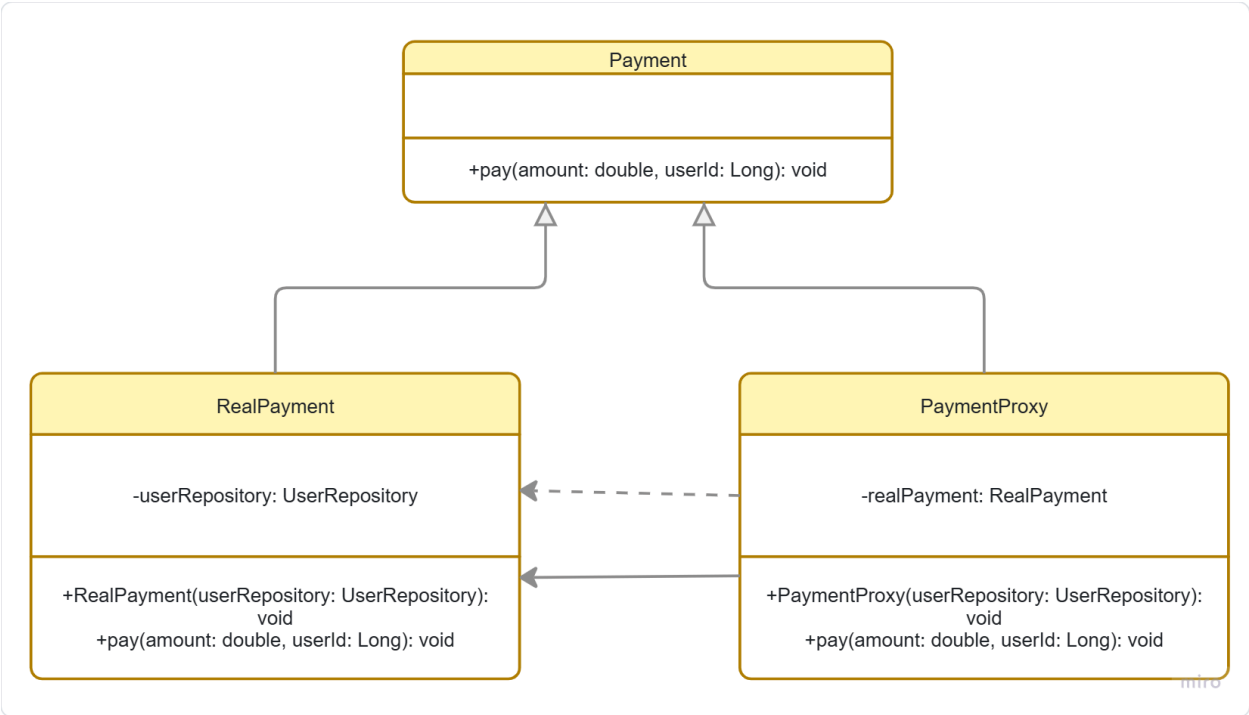
**CarBuilder.java** (abstract builder): Declares the abstract step methods: setMake, setModel, setCarYear, setMileage, setLocation, setPrice, and build. Each returns CarBuilder to support method chaining.
**ConcreteCarBuilder.java** (concrete builder): Extends CarBuilder and owns the Car instance being assembled. Each @Override sets the corresponding field on the Car object and returns this. build() returns the finished Car.
In production, CarController.addCar() uses ConcreteCarBuilder directly with all fields from the incoming CarDTO, then sets ownerId and availability dates separately after the builder completes.
**CarDirector.java:** This constructs a car using the CarBuilder class.



### 2.3.5. Proxy Pattern for Payment Integration

**Payment.java** (interface): Payment is the common interface with a single method.
**RealPayment.java** (real subject): RealPayment is the actual implementation. It looks up the User by ID in the database and deducts the amount from user.balance, then saves the updated user.
**PaymentProxy.java** (proxy): PaymentProxy implements the same Payment interface and holds a reference to a RealPayment instance. Before delegating to realPayment.pay(), it performs a security check: if amount <= 0 it throws a RuntimeException.

**Relationships:**
PaymentService instantiates and uses PaymentProxy

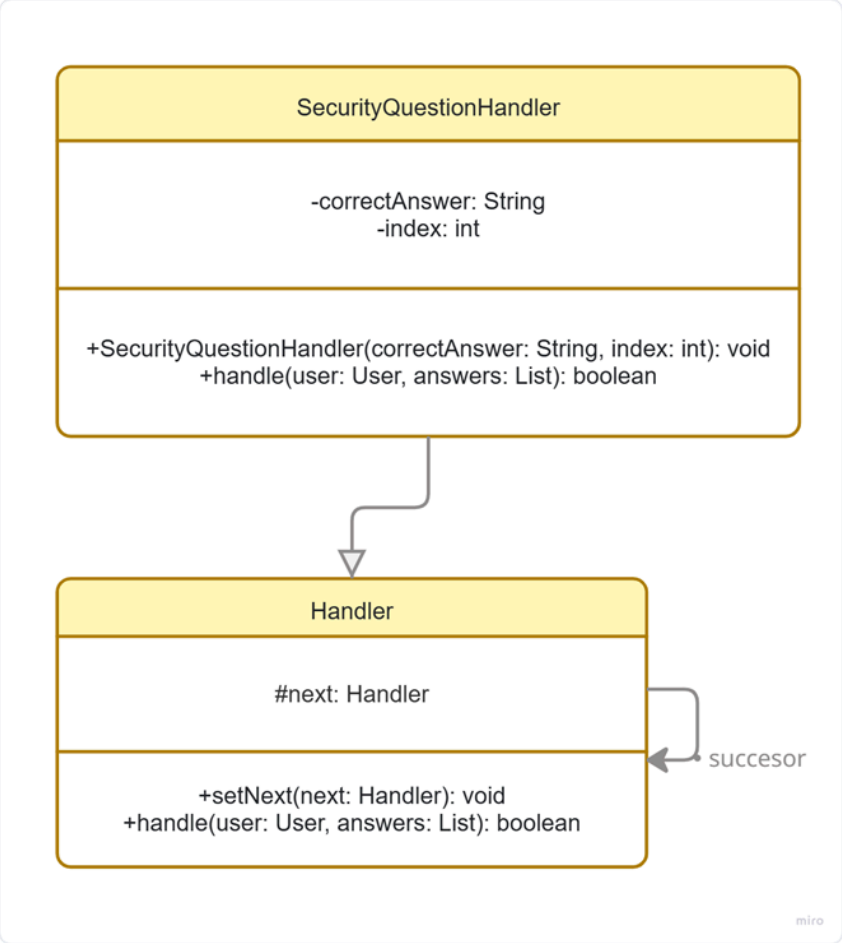


### 2.3.6. Chain of Responsibility for Password Recovery

**Handler.java** (abstract base): Handler is an abstract class with a protected Handler next field and setNext function to build the chain. The abstract handle method must be implemented by subclasses.
**SecurityQuestionHandler.java** (concrete handler): SecurityQuestionHandler stores a correctAnswer string and an index indicating which answer in the submitted list it validates against. In handle(), it compares the submitted answer to its stored correct answer. If they do not match, the chain stops and returns false. If they match, it calls next.handle() to continue down the chain.

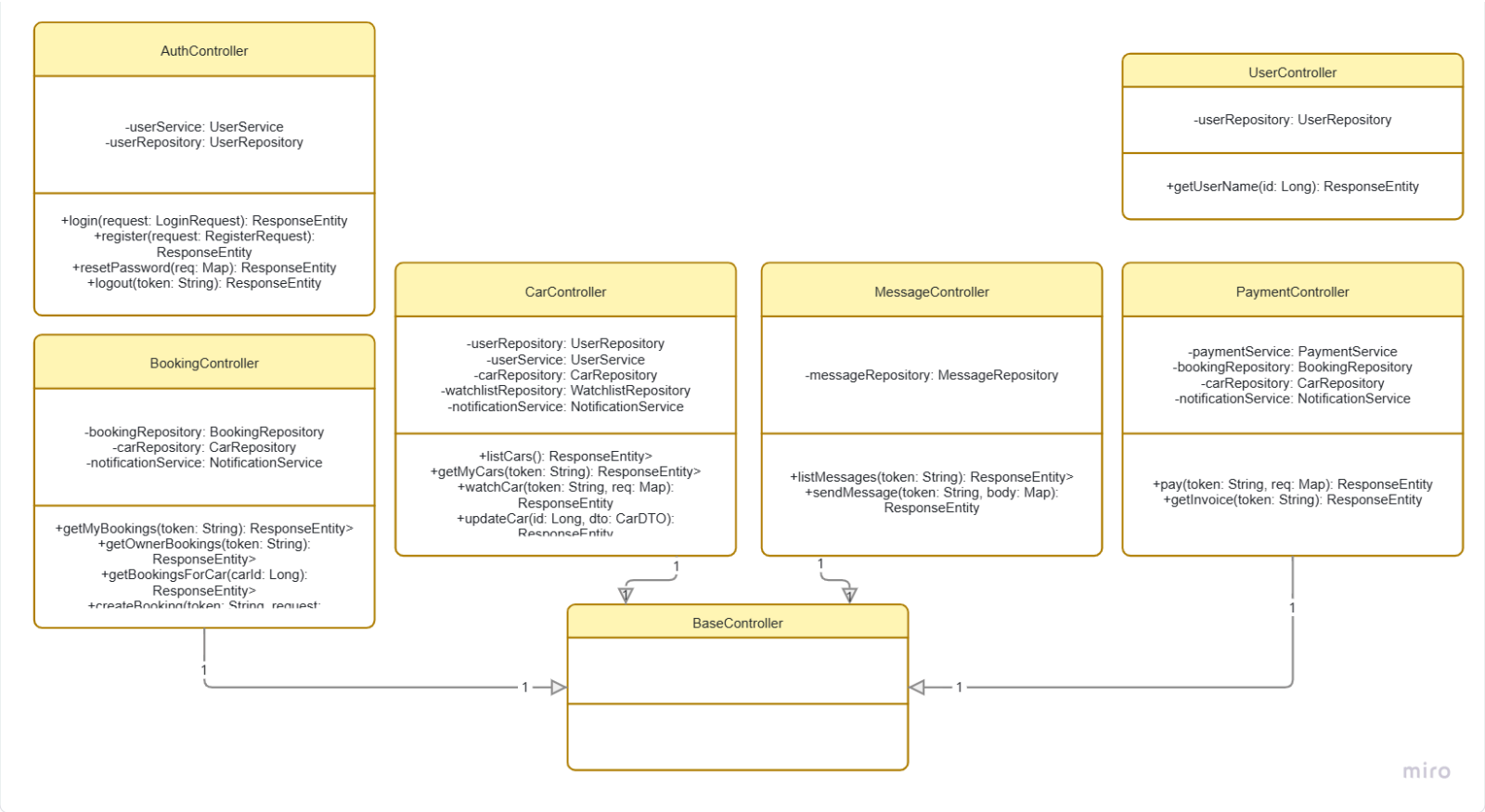
**Relationships:**

- UserService.resetPassword() constructs the chain at runtime: three SecurityQuestionHandler instances are linked together. If all three questions are answered correctly, the password is updated and true is returned. If any question fails, the chain returns false immediately and the password is not changed. UserService builds and executes the chain.
- AuthController calls UserService.resetPassword().



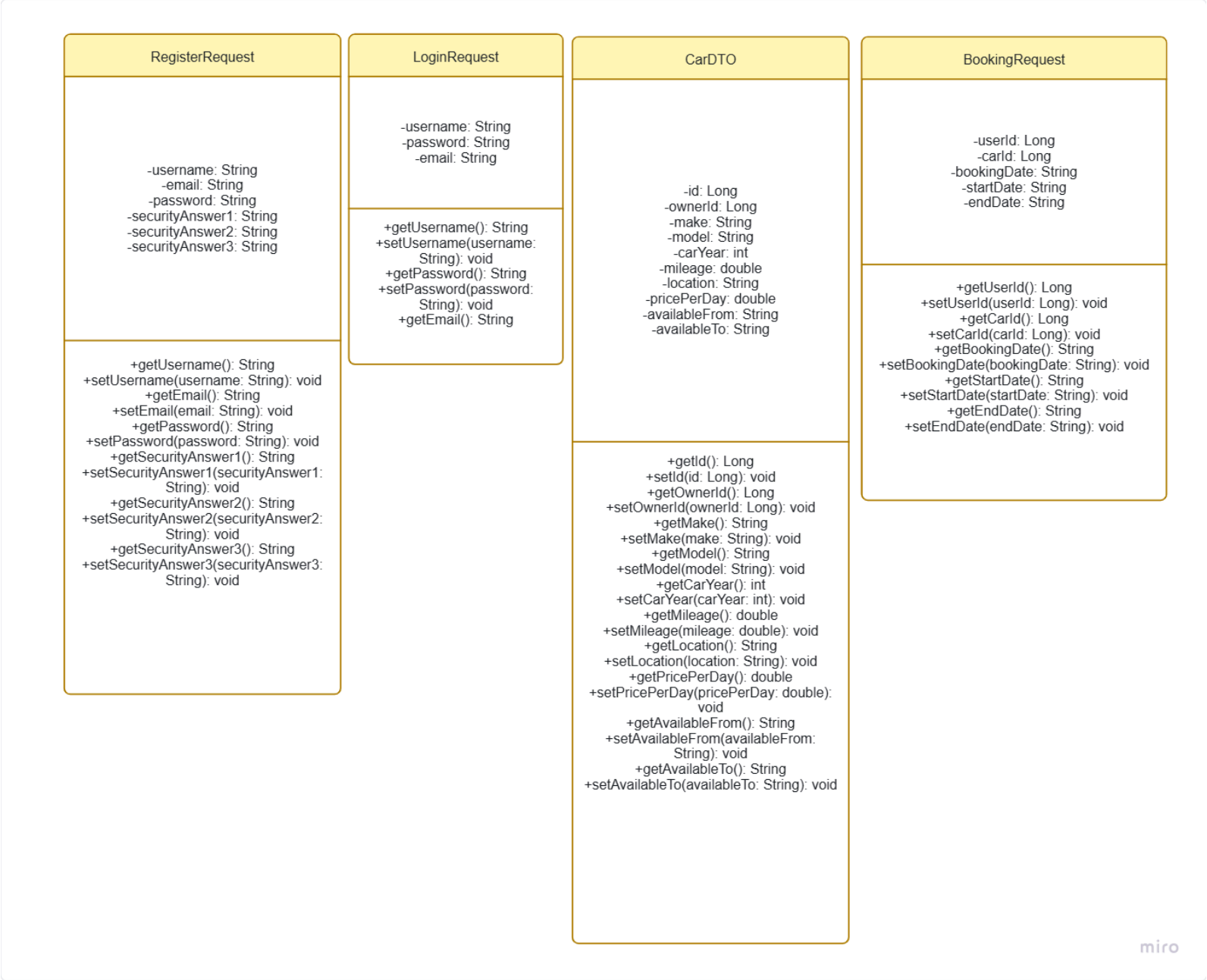
## 2.4. Controller / REST API Layer

**BaseController.java:** Abstract base class extended by all controllers. Provides resolveUser(token) and resolveUserId(token) helpers that look up the logged-in user from the SessionManager singleton using the X-Session-Token header.
**Auth Controller.java:** Handles /api/auth/login, /register, /reset-password, /logout. On login, calls SessionManager.createSession() and returns a UUID token to the browser.
**CarController.java:** Handles /api/cars — list all, list mine (/my), add, update. Triggers Observer notifications on price drops. Saves watchlist entries via WatchlistRepository.
**BookingController.java:** Handles /api/bookings — submit request (POST), get mine (/my), get by car (/car/{id}), get owner requests (/owner), confirm (/{id}/confirm), deny (/{id}/deny). Enforces availability window and conflict checks.
**PaymentController.java:** Handles /api/payment (POST) and /api/payment/invoice (GET). Invoice only includes CONFIRMED+unpaid bookings for the current user. Payment marks them as paid and notifies both parties.
**MessageController.java:** Handles /api/messages — list messages for the current user, send a message. Uses the session token to set senderId.
**UserController.java:** Handles /api/users/{id}/name — returns only a user's id and username. Used by the frontend to display real names in the messaging panel.



## 2.5. DTO

**CarDTO.java:** Transfer object for car data sent to/from the API. Mirrors Car fields including availableFrom/availableTo. Prevents direct entity exposure.
**BookingRequest.java:** Carries the data a renter submits when requesting a booking: userId, carId, startDate, endDate.
**LoginRequest.java:** Carries email and password from the login form to AuthController.
**RegisterRequest.java:** Carries username, email, password, and three securityAnswer fields from the registration form to AuthController.



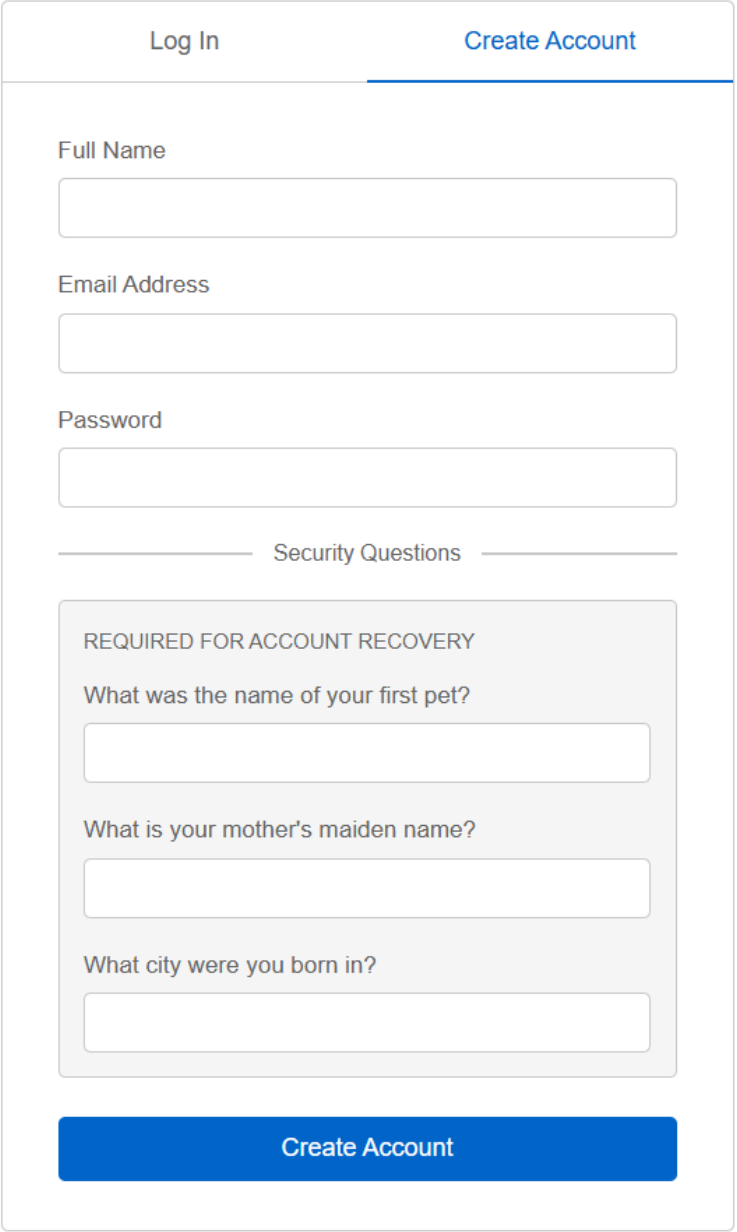
# 3. User-Interface Screenshots

## 3.1. User Registration and Authentication:

Users can register on DriveShare using email and password authentication. During registration, users must set three security questions (with answers).

### 3.1.1. User Registration

User registration screen:



Successful account creation:

Log In | Create Account

Full Name

Asiyah Shoeb

Email Address

asiyah@gmail.com

Password

••••••••••

Security Questions

REQUIRED FOR ACCOUNT RECOVERY

What was the name of your first pet?

bruce

What is your mother's maiden name?

jane

What city were you born in?

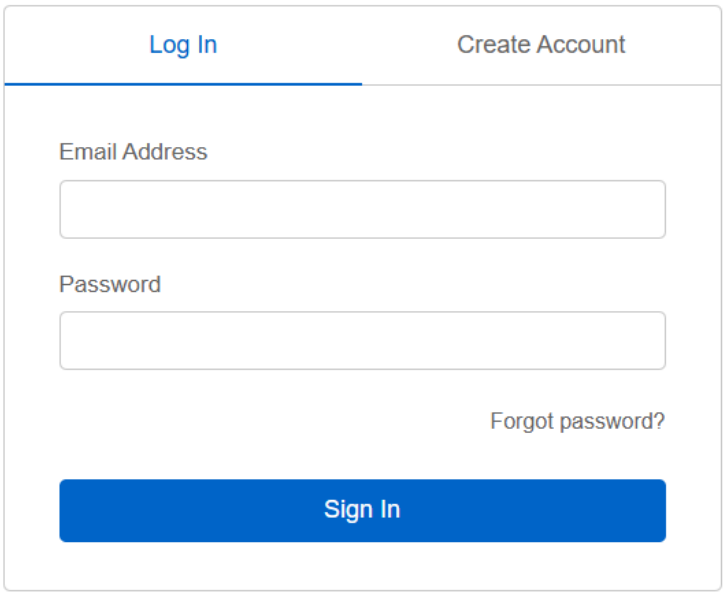
dearborn

Create Account

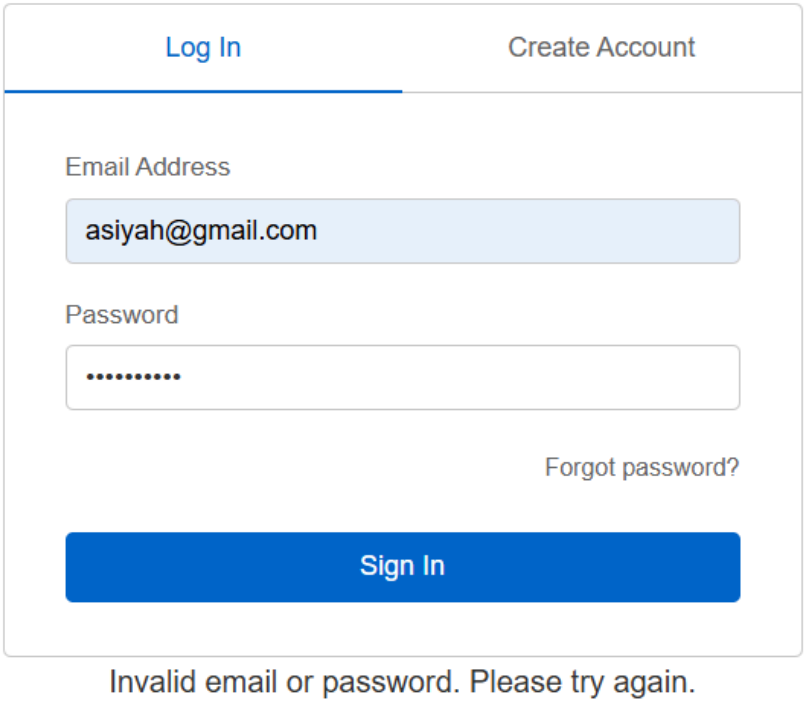
Account created! Please log in.

### 3.1.2. Login

Log in screen:

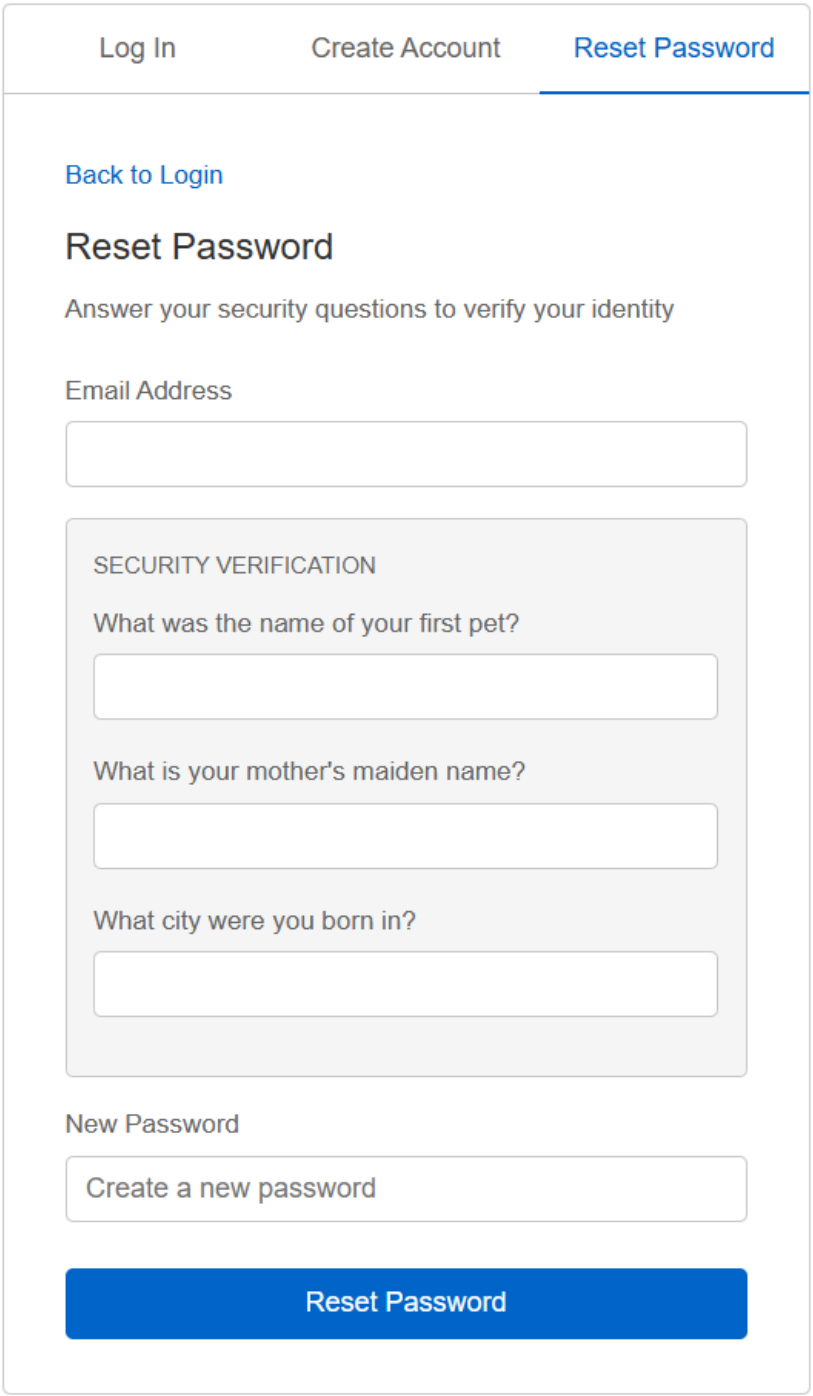



When the user enters invalid credentials, DriveShare displays an error message:

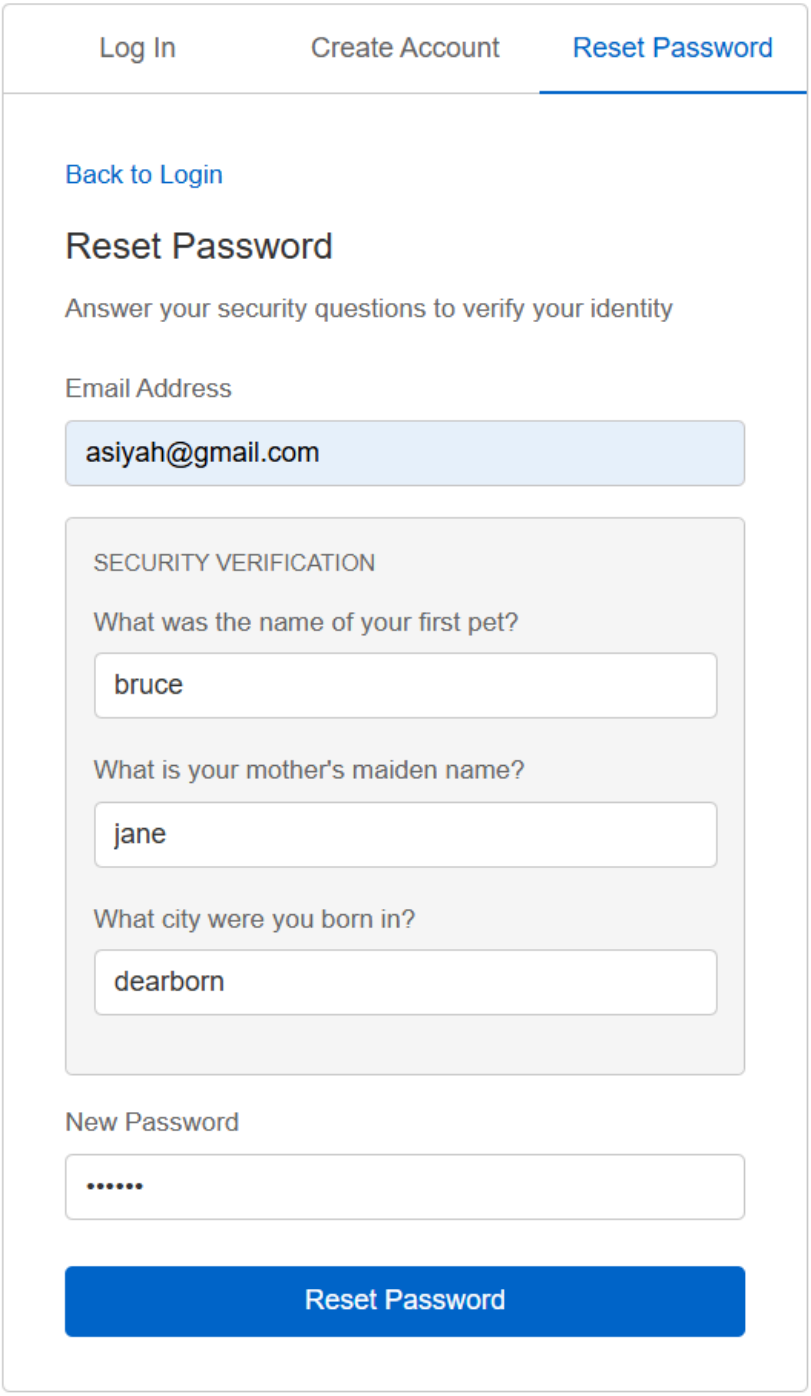


### 3.1.3. Forgot Password

When the user clicks “Forgot password?”, they are redirected to this screen:



Successful password reset when all security questions are answered:




Password reset successful! Please log in.

Unsuccessful password reset due to incorrect answers to security questions:

Log In | Create Account | Reset Password

Back to Login

## Reset Password

Answer your security questions to verify your identity

Email Address

asiyah@gmail.com

SECURITY VERIFICATION

What was the name of your first pet?

bruce

What is your mother's maiden name?

jane

What city were you born in?

wayne

New Password

••••••

Reset Password

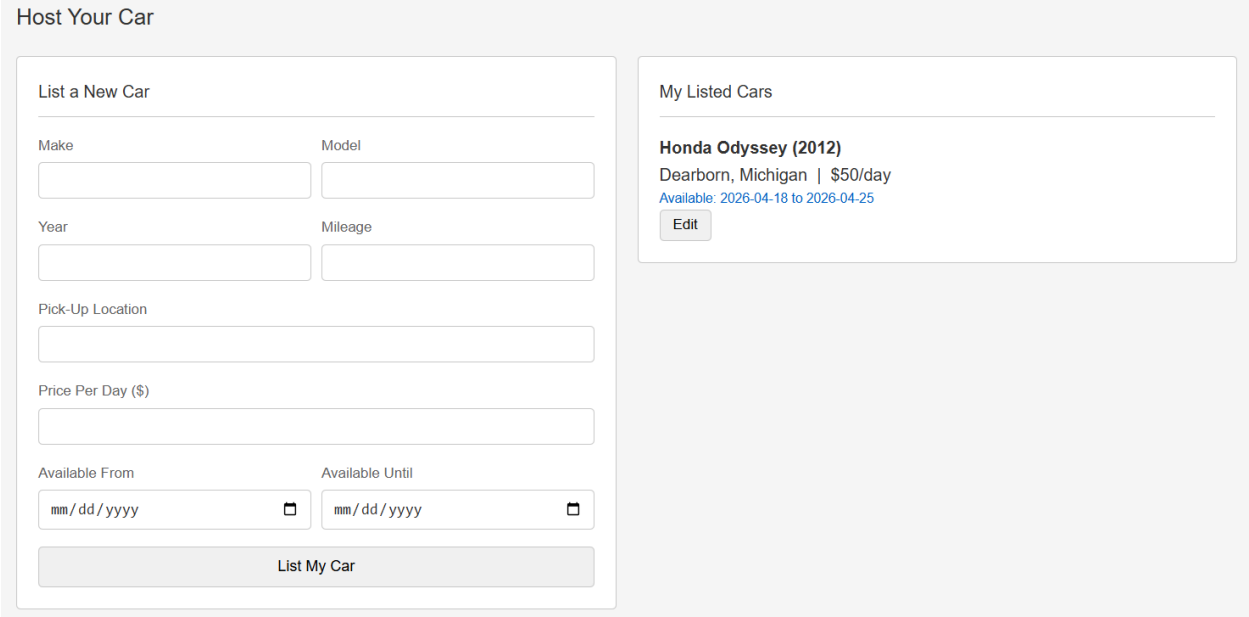
One or more security answers were incorrect.

## 3.2. Car Listing and Management (Owner):

### 3.2.1. Create Car Listing

Car owners can list their vehicles for short-term rental, providing details such as car model, year, mileage, availability calendar, pick up location, and rental pricing.

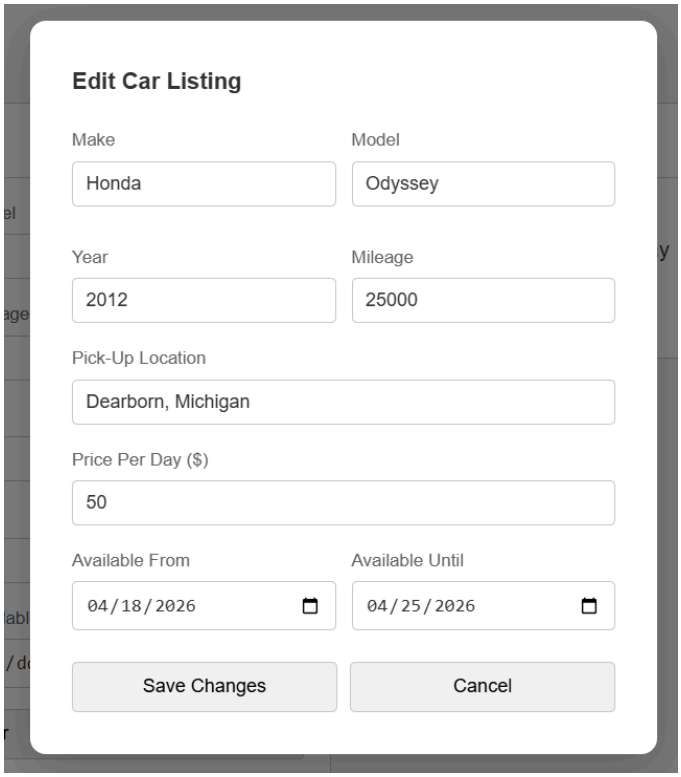
Screen after user enters car listing details and clicks “List My Car”:



### 3.2.2. Edit Car Listing

Owners have the ability to manage their car listings, including updating availability and price.

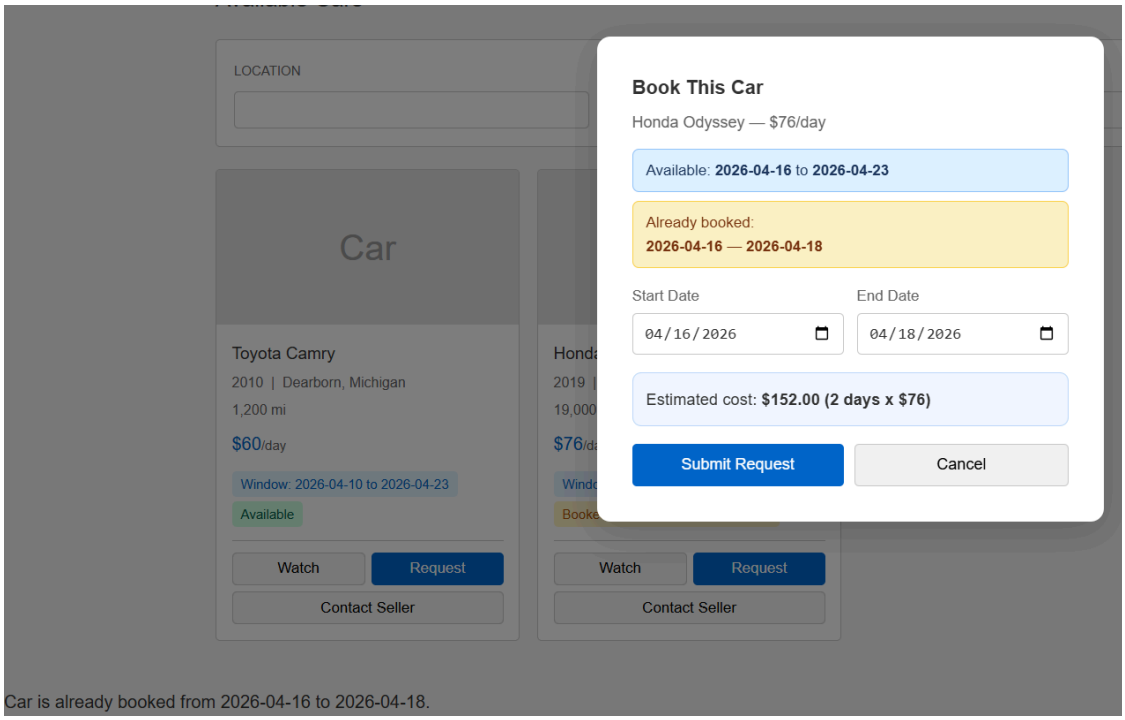
Screen when user clicks “Edit” on desired car listing:



### 3.2.3. Overlapping Dates/Times Prevention

The system must prevent the same car from being rented by more than one renter for overlapping dates/times.

After booking (see section 3.3), the system does not allow another user to request a booking for the same car with overlapping dates (see message on bottom left of screen). User will not be able to “Submit Request” until the dates are changed to not overlap:

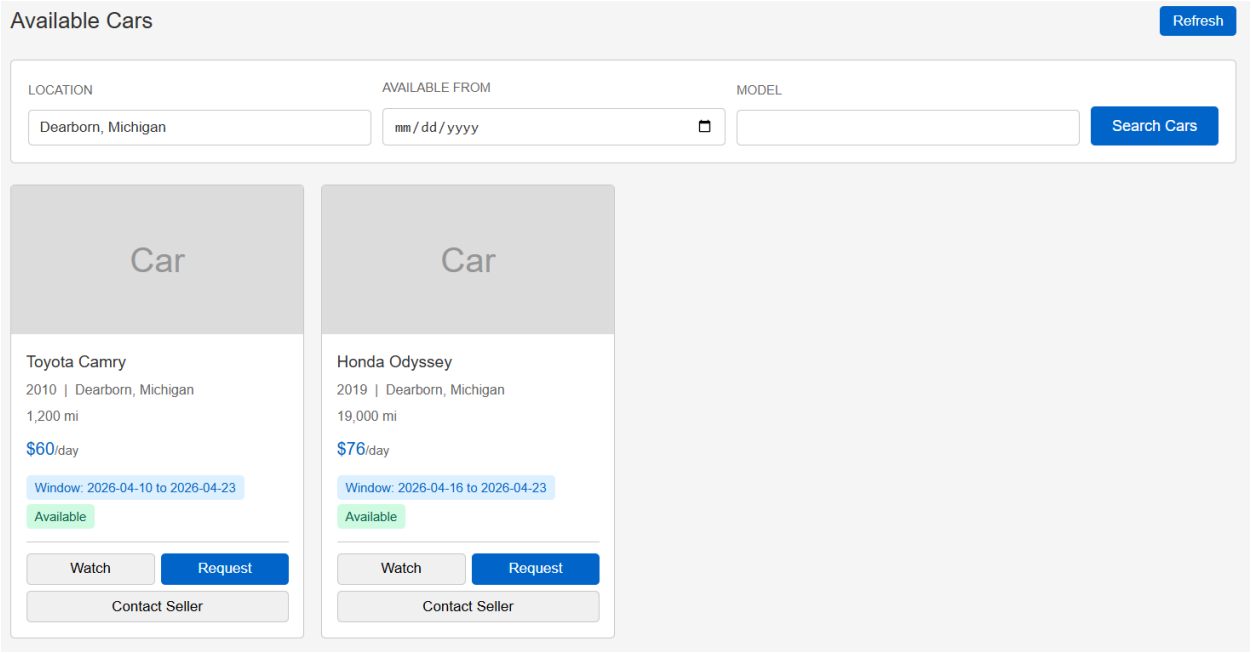


# 3.3 Search and Booking (Renter):

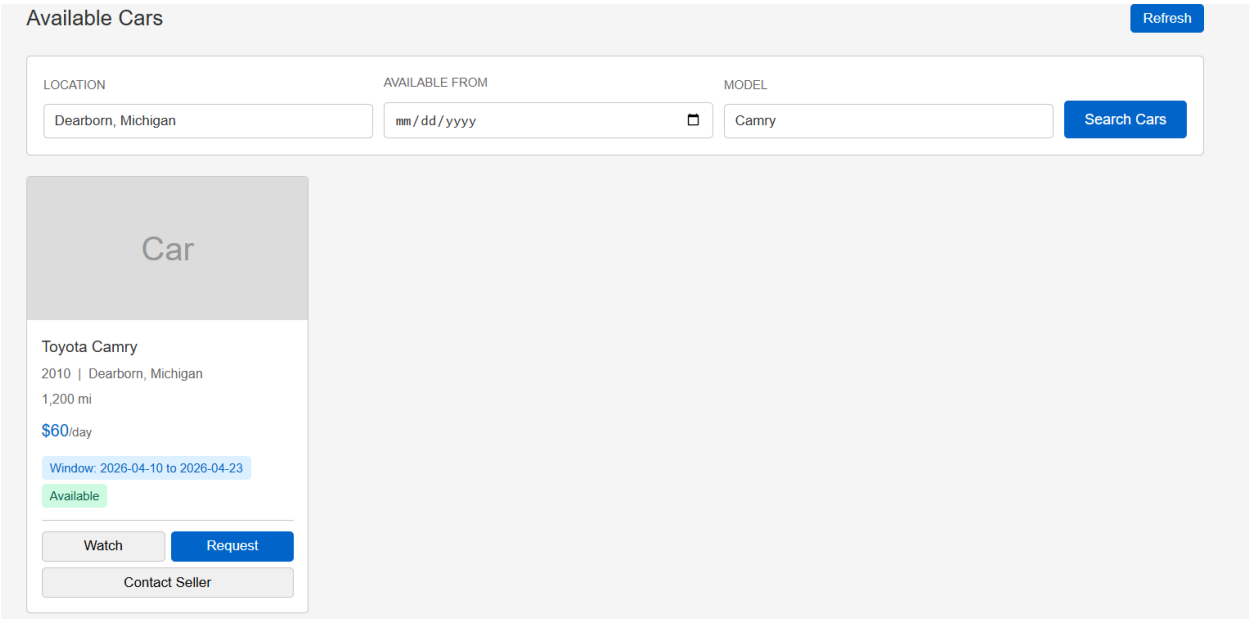
## 3.3.1. Search Car Listings

Renters can search for available cars based on location, date, and other preferences.

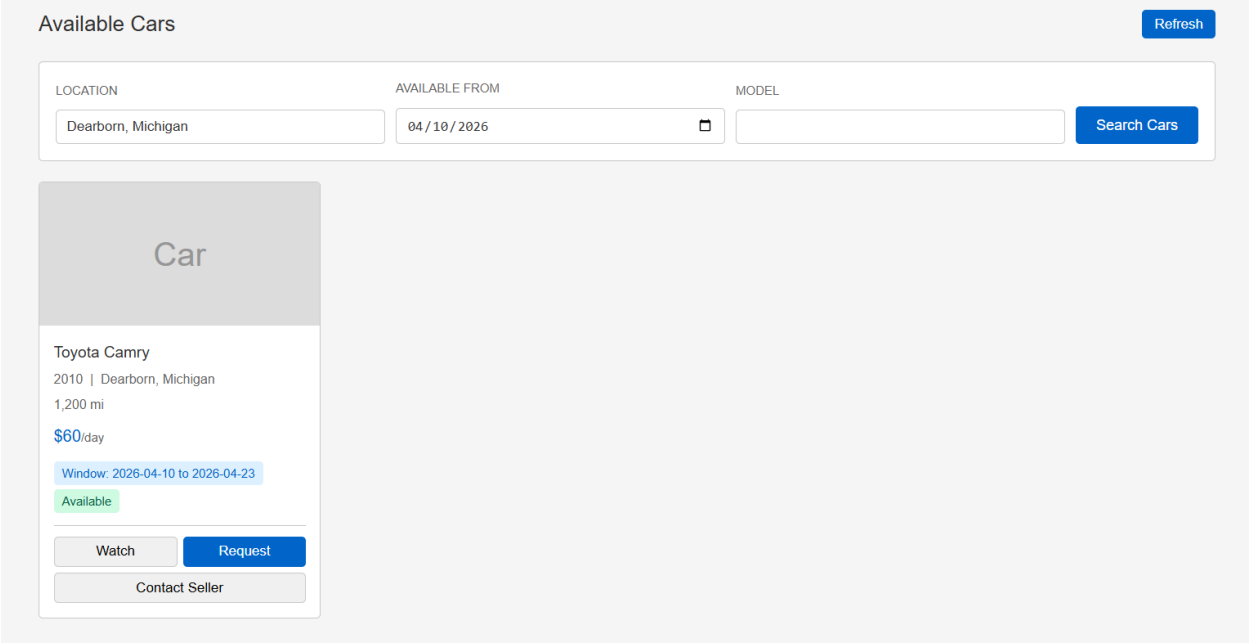
Search based on location:




Search based on location and model:



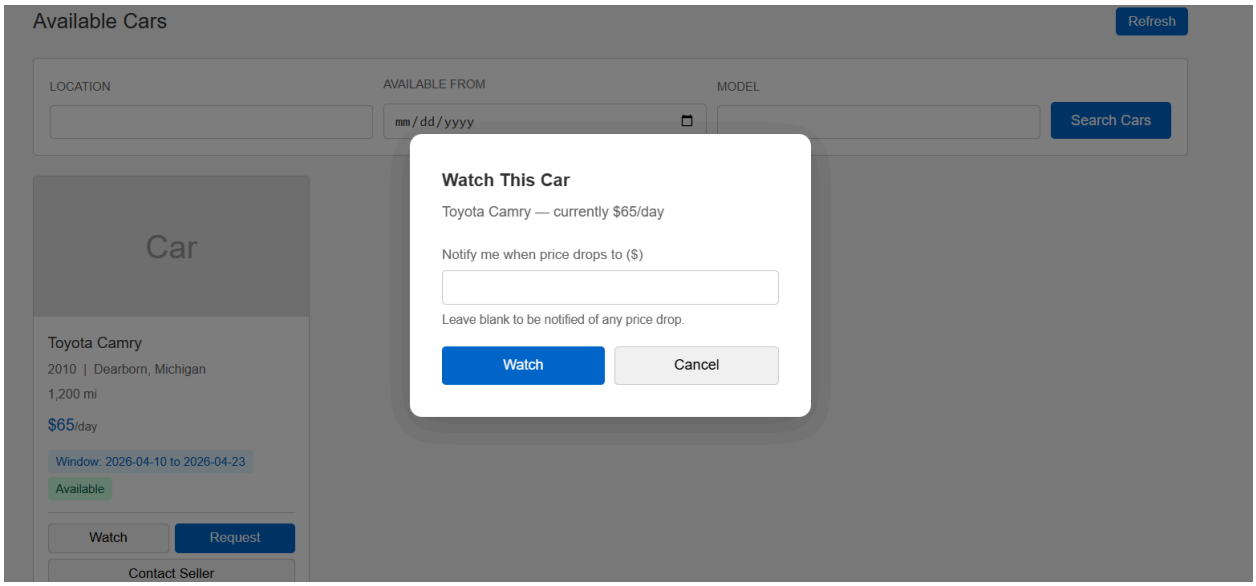
Search based on location and availability date:



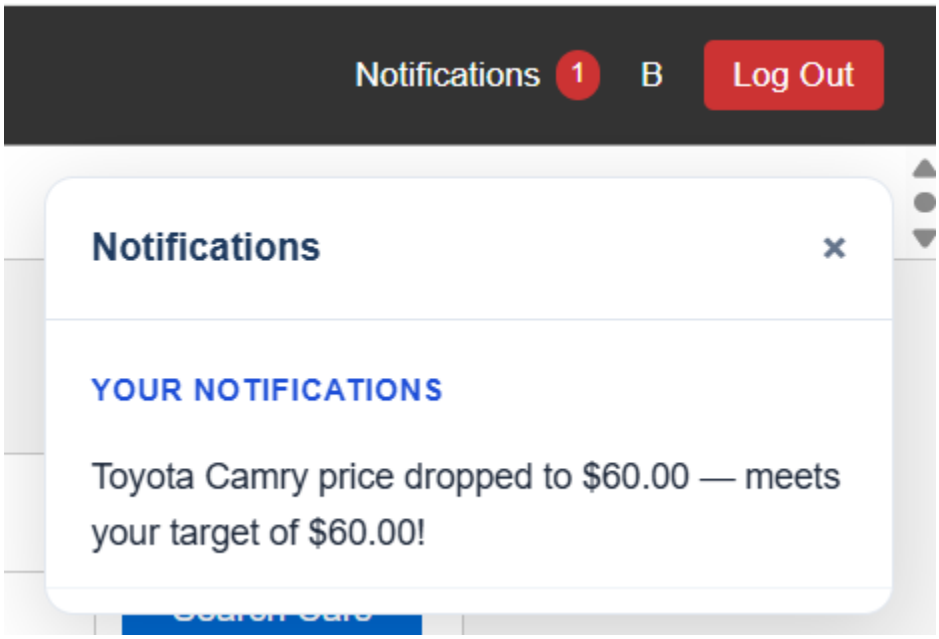
### 3.3.2. Watch Car Listings

Renters can “watch” desired cars and receive notification when the car becomes available and/or rental prices drop to meet renter-defined specifications.

After clicking “Watch” on the car listing, the user is prompted to enter their desired price:



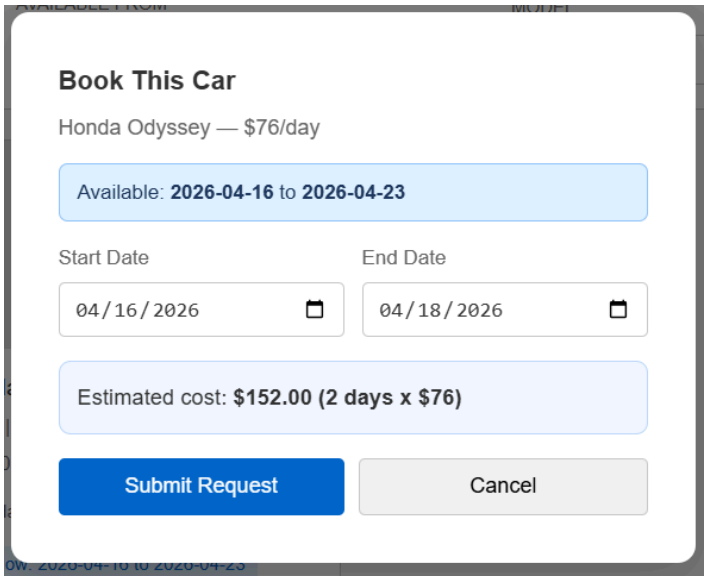

User is notified when price matches their desired price:



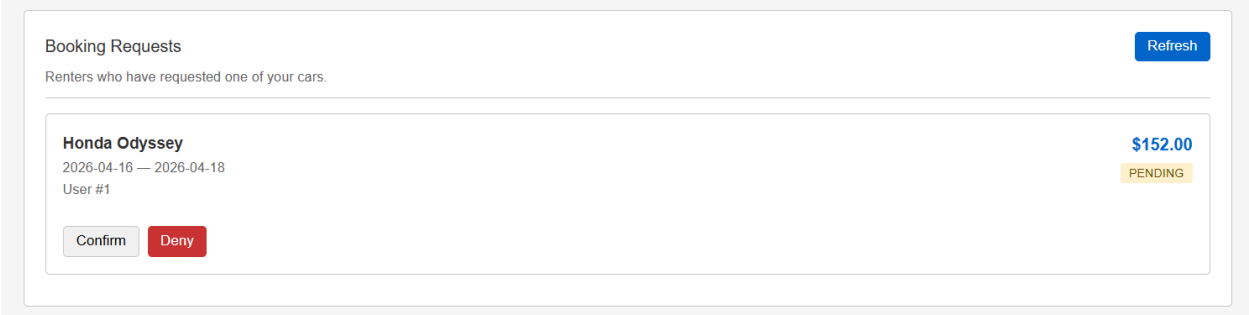
### 3.3.3. Reserve Car

Implement a booking system allowing renters to reserve a car for a specific period.

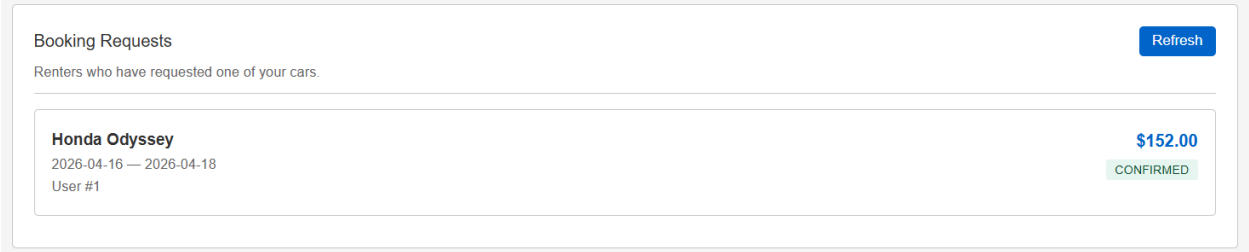
After clicking “Request” on the desired car listing, buyer is prompted to enter desired time period in this screen:



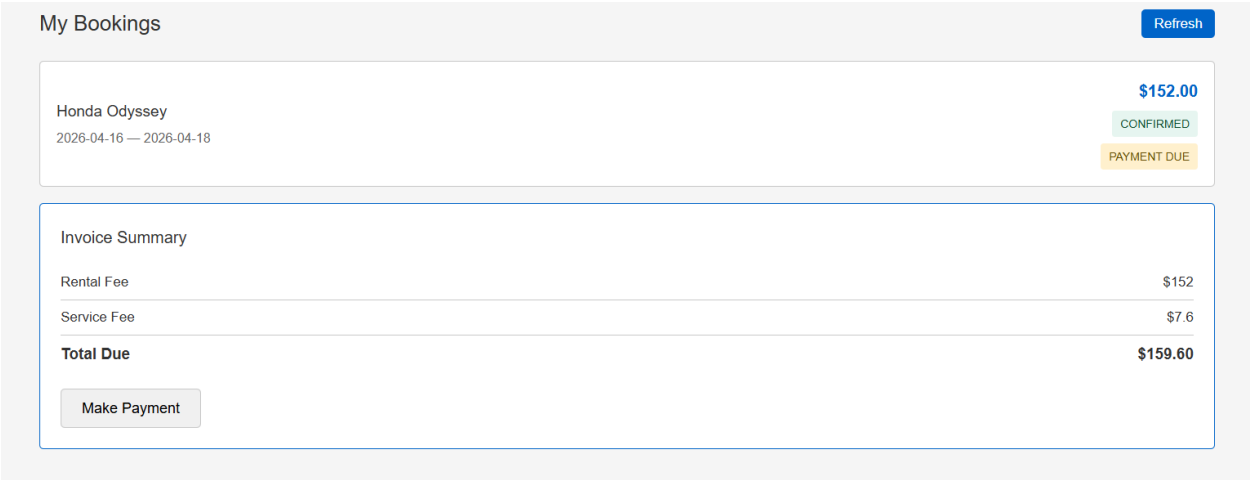

Upon the buyer clicking “Submit Request”, the owner can now view the booking request in their “Host Your Car” tab and can confirm or deny the request.




Screen after buyer confirms booking:



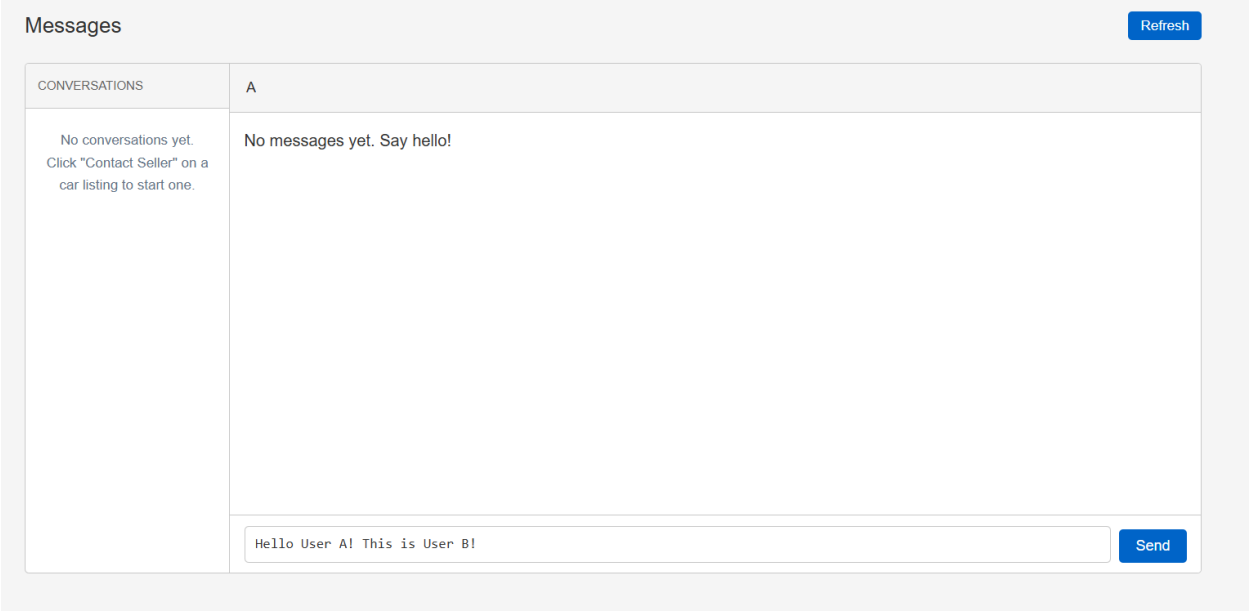
After the seller confirms the booking, the booking appears in the buyer's "My Bookings" tab with status "confirmed":



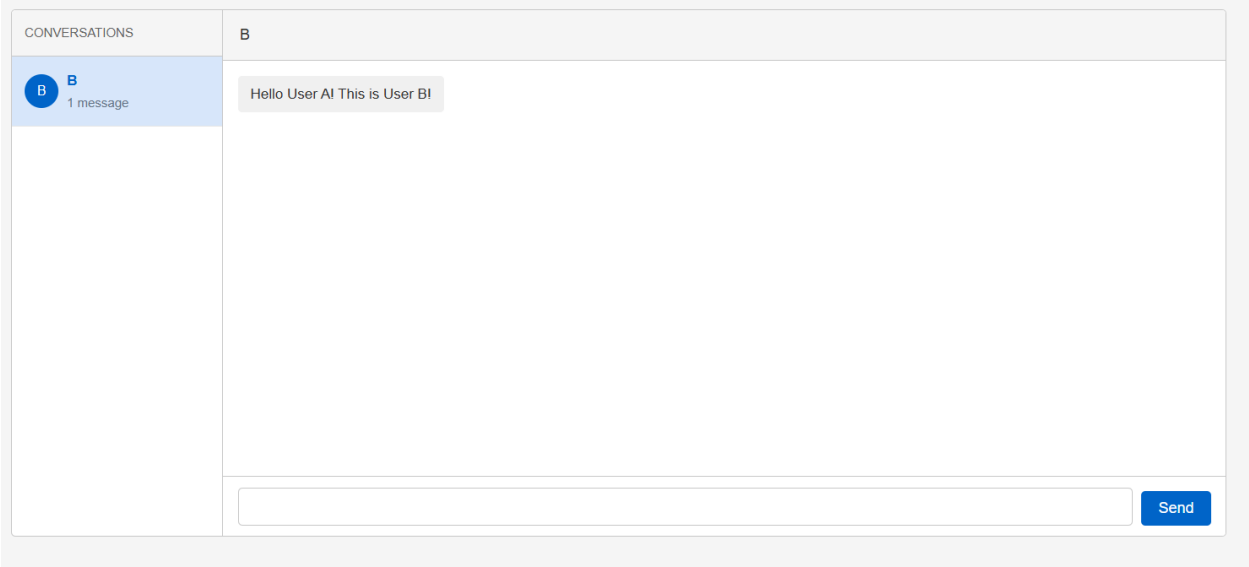
# 3.4. Messaging and Communication

## 3.4.1. Messaging System

DriveShare allows users to message each other. In this example, user B has clicked “Contact Seller”:



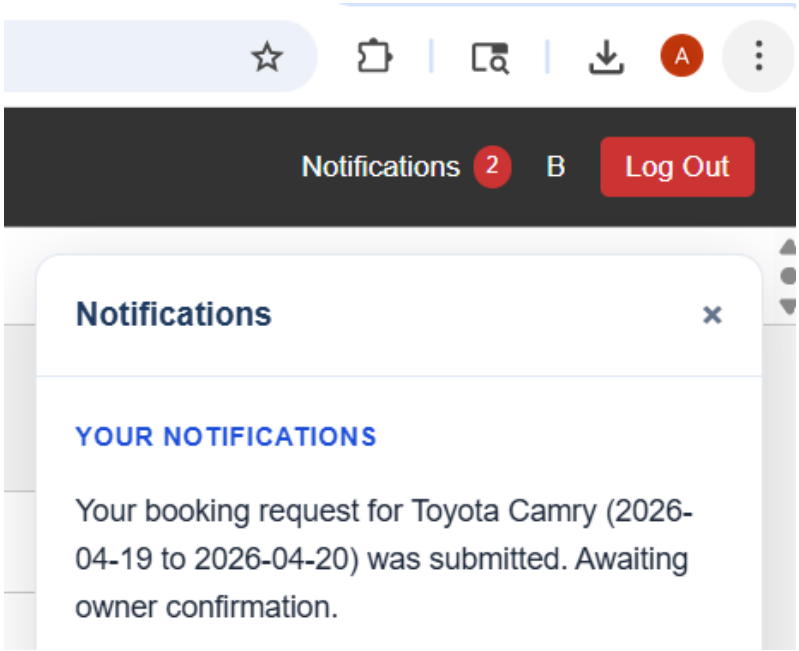

User A’s screen after User B has sent the message:



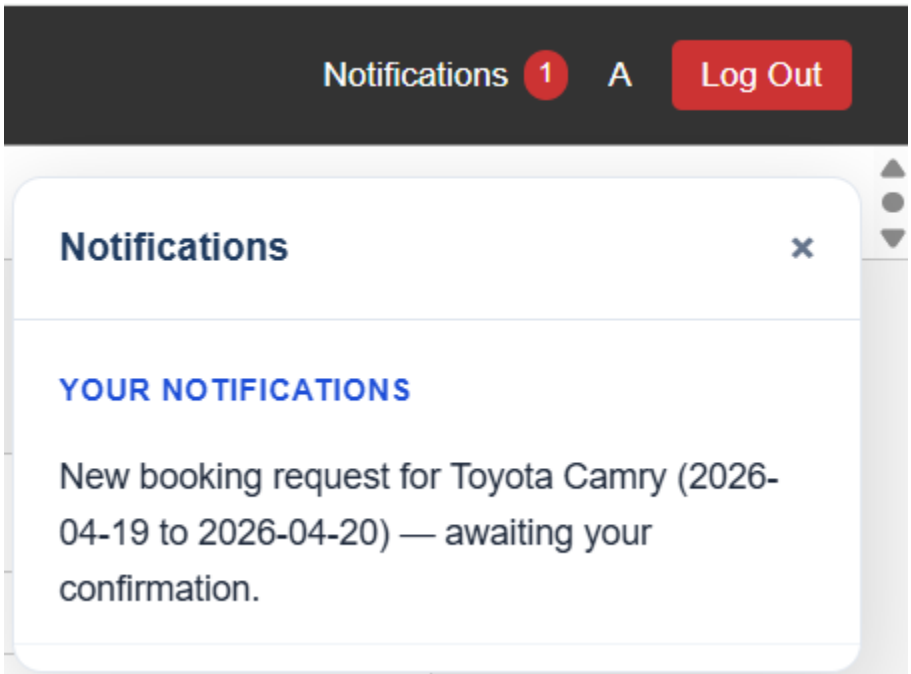
### 3.4.2. Notification System

DriveShare will send notifications for booking requests, confirmations, and important updates via email or in-app messages.

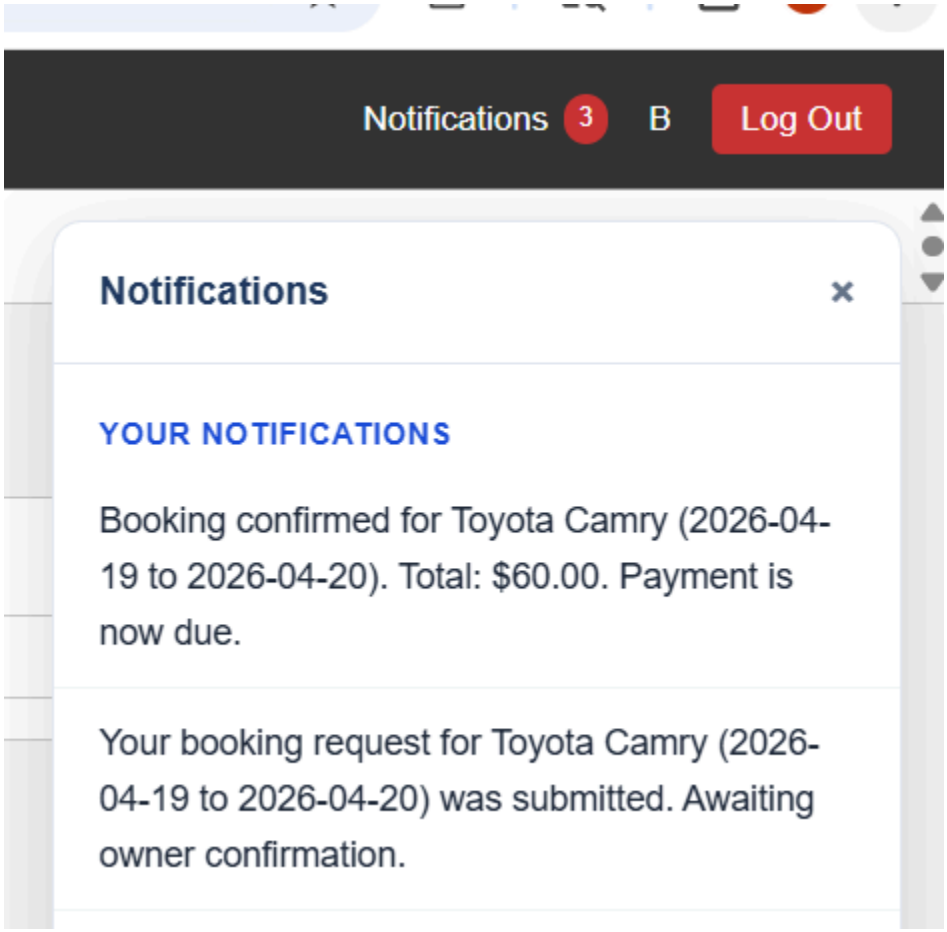
Notification to the buyer upon submitting the booking request:




Notification to the owner when there is a booking request:



Notification to buyer when booking request has been confirmed:

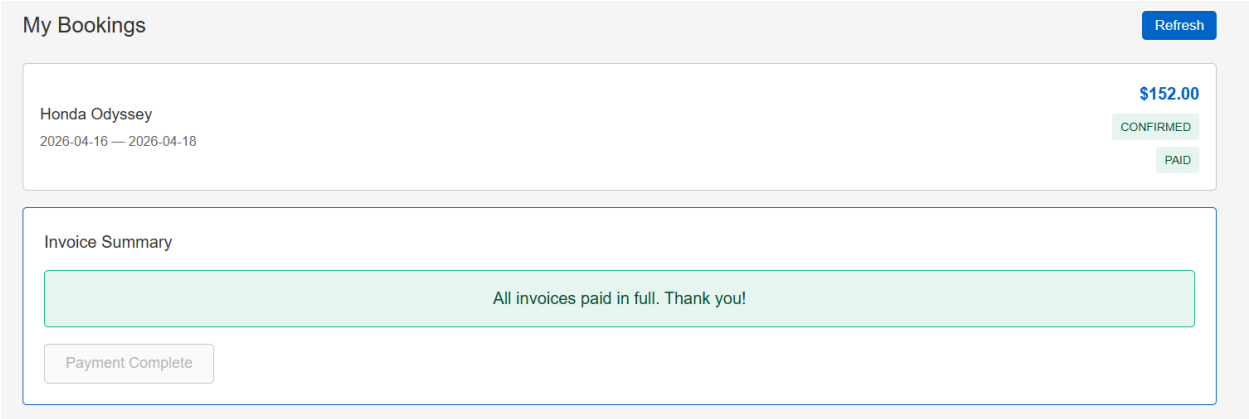



See notifications for payment updates in Section 3.5 and notifications for price changes in cars being “watched” in Section 3.3.2.

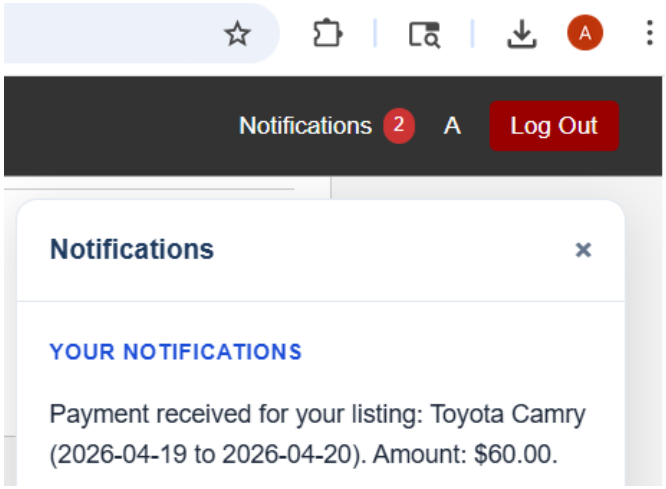
## 3.5. Payment

DriverShare allows renters to make payments:

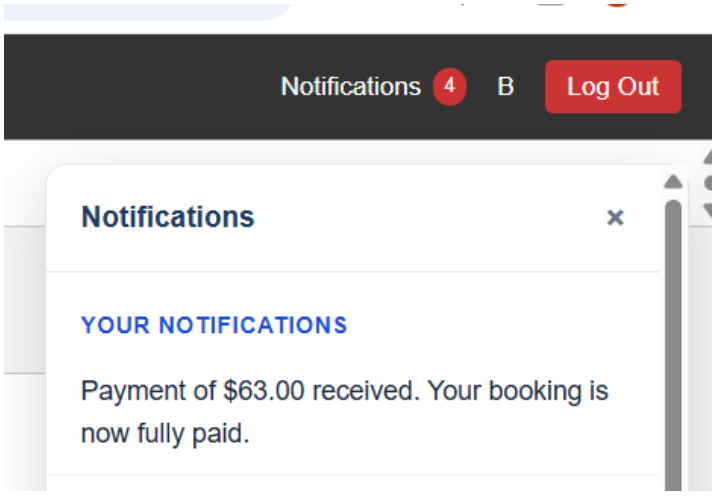
Screen after successful payment:




Notification to Seller:




Notification to Buyer:



# 4. References

The Apache Software Foundation. “Maven Documentation – Maven.” *Apache Maven*, 2026, https://maven.apache.org/guides/index.html.

Oracle. “Learn Java.” 2026, https://dev.java/learn/.

Spring. “Spring Boot Reference Documentation.” 2023, https://docs.spring.io/spring-boot/docs/2.7.15/reference/html/.

Spring. “Spring Data JPA.” 2008, https://docs.spring.io/spring-data/jpa/reference/index.html.

Lecture slides and course materials.